Edge cases (a missing required argument) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `missing_argument_practice.py`: when a required argument is missing from the user's message, the model can ask, guess, or send an empty value — and which one it does tells you whether your tool needs to design for a clarifying question or a graceful default. **If not:** the Build Task's Pydantic argument models would be guessing at a problem you'd never actually watched happen.

Basic Version

Approach 1 — the direct way

```
# missing_argument_practice.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a named city. Requires a city name."""
    return f"Sunny in {city}"

model_with_tools = ChatOpenAI().bind_tools([get_weather])

for run_number in range(1, 6):
    response = model_with_tools.invoke("what's the weather like")

    if not response.tool_calls:
        reply = response.content
        print(f"Run {run_number}: asked or answered directly -> {reply}")
        continue

    city_sent = response.tool_calls[0]["args"].get("city")
    if not city_sent:
        print(f"Run {run_number}: called with an empty city value")
```

```
    else:
        print(f"Run {run_number}: called with city = {city_sent}")
```

Expected output (an example split — real model behavior varies run to run):

```
Run 1: asked or answered directly -> Sure — which city do you mean?
Run 2: called with city = London
Run 3: asked or answered directly -> Could you tell me which city you mean?
Run 4: called with city = New York
Run 5: called with an empty city value
```

This shows you the real split across 5 runs. It doesn't save the results anywhere, so once the run finishes you've only got what scrolled past in the terminal — fine for a quick look, not for writing up what you found.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — classified and returned as structured records

```
# missing_argument_practice.py
from langchain_core.runnables import Runnable
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI


@tool
def get_weather(city: str) -> str:
    """Get the current weather for a named city. Requires a city name."""
    return f"Sunny in {city}"


def get_model_with_tools() -> Runnable:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return model.bind_tools([get_weather])


def classify_response(response) -> str:
    # why: sorts each run into one of 3 real outcomes instead of leaving it
    # to a human reading scrolled-past terminal text.
    if not response.tool_calls:
        return "asked_or_answered_directly"
    city_sent = response.tool_calls[0]["args"].get("city")
    if not city_sent:
        return "called_with_empty_value"
    return "called_with_guessed_value"


def run_missing_argument_trial(prompt: str, runs: int = 5) -> list[dict]:
    model_with_tools = get_model_with_tools()
    records = []

    for run_number in range(1, runs + 1):
        response = model_with_tools.invoke(prompt)
        outcome = classify_response(response)
        record = {"run": run_number, "outcome": outcome}
        if outcome == "called_with_guessed_value":
```

```
            record["city_sent"] = response.tool_calls[0]["args"].get("city")
        if outcome == "asked_or_answered_directly":
            record["reply"] = response.content
        records.append(record)

    return records


if __name__ == "__main__":
    results = run_missing_argument_trial("what's the weather like")
    for record in results:
        print(record)
```

Expected output (example):

```
{'run': 1, 'outcome': 'asked_or_answered_directly', 'reply': 'Which city?'}
{'run': 2, 'outcome': 'called_with_guessed_value', 'city_sent': 'London'}
{'run': 3, 'outcome': 'asked_or_answered_directly', 'reply': 'Specify a city?'}
{'run': 4, 'outcome': 'called_with_guessed_value', 'city_sent': 'New York'}
{'run': 5, 'outcome': 'called_with_empty_value'}
```

Difference from Basic: returning a list of typed records instead of only printing lets you actually count the split afterward (how many asked vs. guessed vs. something else) rather than having to re-read scrolled-past terminal output. Separating `classify_response()` out also makes it independently testable against a fake `response` object, no real API call needed.

Approach 2 — `args_schema` rejects an empty city before the tool runs

Story — `WeatherArgs`: Approach 1 only *watches* the model send an empty city. A Pydantic model attached with `args_schema=` makes the tool itself refuse it — the "Checking arguments: Pydantic as the contract" topic, used for real. **If not:** `get_weather("")` would happily answer "Sunny in " — a wrong result that looks like a real one.

```
# missing_argument_practice.py
from pydantic import BaseModel, Field, ValidationError
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI


class WeatherArgs(BaseModel):
    # why: min_length=1 — an empty city fails this check before the
    # tool's own code ever runs
    city: str = Field(min_length=1)
```

```
# how: args_schema= makes the tool check its arguments with WeatherArgs
@tool(args_schema=WeatherArgs)
def get_weather(city: str) -> str:
    """Get the current weather for a named city. Requires a city name."""
    return f"Sunny in {city}"


def run_tool_safely(args: dict) -> str:
    try:
        return get_weather.invoke(args)
    except ValidationError:
        # why: a clear message the model can read — and then ask the user
        # which city — instead of a crash
        return "Error: a city name is required - ask the user."


def main() -> None:
    good_args = {"city": "Lahore"}
    empty_args = {"city": ""}
    print(f"case 1 (city given)   -> {run_tool_safely(good_args)}")
    print(f"case 2 (empty city)   -> {run_tool_safely(empty_args)}")

    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools([get_weather])
    response = model_with_tools.invoke("what's the weather like")
    if not response.tool_calls:
        print(f"case 3 (real model)   -> asked: {response.content}")
    else:
        result = run_tool_safely(response.tool_calls[0]["args"])
        print(f"case 3 (real model)   -> {result}")


if __name__ == "__main__":
    main()
```

Expected output (case 3 depends on the model):

```
case 1 (city given)   -> Sunny in Lahore
case 2 (empty city)   -> Error: a city name is required - ask the user.
case 3 (real model)   -> asked: Which city do you mean?
```

Difference from Approach 1: Approach 1 observes and labels what the model does. Approach 2 fixes it at the tool: `Field(min_length=1)` rejects an empty city with a `ValidationError`, and `run_tool_safely()` turns that into a message the model can read and act on (ask the user) instead of a crash.

Which one should you actually write? Both — Approach 1 to see the problem, Approach 2 to fix it. The Build Task gives every tool an `args_schema` model and catches `ValidationError` exactly like `run_tool_safely()`.